

PROJECT 6

Vehicle Maintenance Prediction & Agentic Fleet Management

From Predictive Maintenance to Autonomous Fleet Operations

Team Members:

Milind Bansal

Isha Tomar

Yashi Agrawal

Mansi Agarwal

Section: D

Faculty: Kartik Gupta

April 18, 2026

Abstract

Modern fleet operations generate continuous streams of operational telemetry data including mileage, engine load, vibration, temperature, and environmental conditions. However, maintenance decisions are frequently based on fixed schedules or reactive breakdown-based strategies. These approaches fail to incorporate real-time operational stress and vehicle-specific conditions.

This project presents an end-to-end supervised machine learning framework for predicting whether a vehicle requires maintenance. Unlike traditional binary classification problems, this task represents a risk-detection scenario where false negatives may result in breakdowns, downtime, financial loss, and safety hazards. Therefore, evaluation prioritizes recall, F1-score, and ROC-AUC over simple accuracy.

The dataset consists of 15,000 structured vehicle records with 28 engineered and encoded features. Extensive exploratory data analysis, correlation study, outlier handling, and feature engineering were conducted before model training. Four classification models were implemented and compared: Logistic Regression, Decision Tree, Random Forest, and XGBoost.

After hyperparameter optimization using `RandomizedSearchCV`, the tuned XGBoost classifier achieved an ROC-AUC of 0.92 and balanced minority-class recall performance.

To transition from predictive analytics to prescriptive operational intelligence, the deployment architecture securely incorporates an autonomous Agentic Fleet Management system. Using `LangGraph`, this agent integrates the machine learning predictions with Retrieval-Augmented Generation (RAG), rule-based diagnostics, and a Large Language Model (LLM) to generate actionable and contextual maintenance plans via an interactive Streamlit framework.

Ultimately, this work demonstrates a complete, production-grade intelligence pipeline bridging the gap between data preprocessing and autonomous decision execution.

0 Contents

1	Introduction	4
2	Dataset Description	4
3	Exploratory Data Analysis	5
3.1	Feature Scale Variation	5
3.2	Outlier Detection	5
3.3	Class Imbalance	5
4	Feature Engineering	7
4.1	Rationale for Feature Engineering	7
4.2	Engineered Features	7
4.3	Validation Through Feature Importance	7
5	Model Training and Selection Strategy	8
5.1	Train-Test Split	8
5.2	Progressive Model Evaluation	8
5.3	Hyperparameter Optimization Strategy	8
5.4	Model Comparison Summary	9
6	Agentic Fleet Management Architecture	10
6.1	Agent Components and State Graph Integration	10
6.2	LLM Orchestration and Generative Enrichment	11
7	End-to-End System Workflow	12
7.1	Actual Execution Flow	12
8	Why This Architecture Matters	12
9	Agent Output Example	12
10	Case Study: Direct Prediction vs Agent Interpretation	13
11	Implementation Summary	14
12	Limitations and Future Work	14
13	Repository Architecture	15
14	Technology Stack	15

15 Conclusion	15
16 Team Contribution	16
17 References	16

1 Introduction

Fleet maintenance is traditionally performed either at fixed service intervals or after mechanical failure occurs. Fixed scheduling ignores real operational stress, while reactive maintenance increases downtime and operational risk.

The core predictive problem addressed in this project is:

Can historical vehicle telemetry data be used to proactively predict whether a vehicle requires maintenance?

This is not merely a classification problem; it is a risk-sensitive detection problem. A false negative — failing to detect required maintenance — may result in:

- Unexpected mechanical breakdown
- Operational downtime
- Increased repair costs
- Safety hazards

Beyond prediction, this project formally addresses the prescriptive challenge: *what action should be taken when a risk is detected?* By embedding the predictive ML model into a stateful, LLM-powered agent workflow, the system moves beyond merely predicting failure. It actively and autonomously diagnoses subsystems, interprets recurring variables, and outputs precise maintenance procedures.

2 Dataset Description

The dataset consists of structured vehicle telemetry features collected from simulated fleet operations.

Table 1: Dataset Overview

Property	Value
Total Samples	15,000
Target Variable	Maintenance Required (0/1)
Class Distribution	75% No Maintenance, 25% Maintenance
Raw Numerical Features	10
Categorical Features	5
Total Features After Encoding	28

3 Exploratory Data Analysis

3.1 Feature Scale Variation

The statistical summary revealed significant variation in feature magnitudes. For example:

- Mileage ranged in tens of thousands.
- Battery voltage ranged within a narrow band.
- Vibration levels were tightly distributed.

This influenced the need for scaling in linear models during preprocessing.

3.2 Outlier Detection

Boxplot analysis revealed extreme upper tails in mileage and engine hours. Instead of removing these records, IQR-based capping was applied because high-usage vehicles represent realistic operational scenarios.

3.3 Class Imbalance

The dataset exhibited moderate imbalance (75:25). Accuracy alone would therefore be misleading. For example, predicting all vehicles as "No Maintenance" yields 75% accuracy without predictive power. Thus, Recall and F1-score were heavily prioritized throughout the model evaluation.

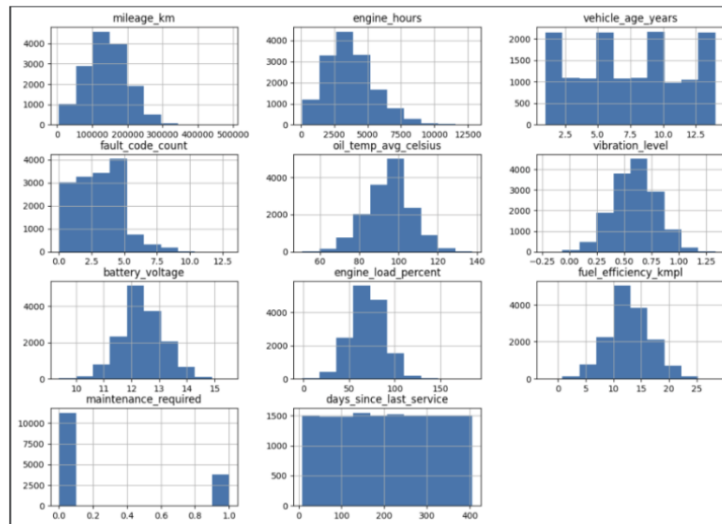


Figure 1: Distribution and Values Range for Telemetry Variables

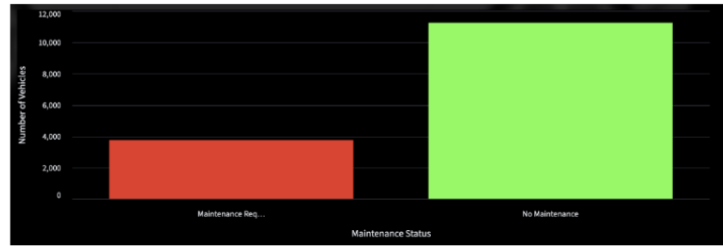


Figure 2: Class Imbalance in Target Variable

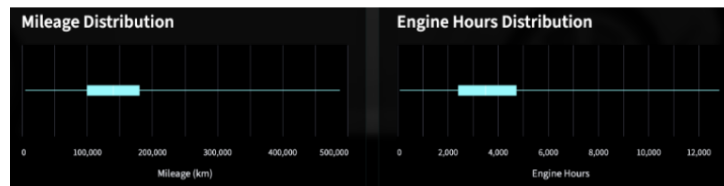


Figure 3: Box Plot indicating underlying distributions and bounds

4 Feature Engineering

Raw telemetry features often fail to capture the interaction effects and operational intensity patterns that influence mechanical degradation. Therefore, domain-informed feature engineering was performed within the `utils/preprocessor.py` logic to transform primary signals into more meaningful stress indicators.

4.1 Rationale for Feature Engineering

Vehicle maintenance requirements are rarely triggered by a single variable in isolation. Instead, risk emerges from compounded operational stress, long-term wear, and contextual environments. For example:

- High mileage accumulated slowly over 10 years does not represent the same stress as high mileage accumulated within 2 years.
- High engine load under normal temperature may be acceptable, but high load combined with elevated temperature significantly accelerates thermal wear.

4.2 Engineered Features

Table 2: Engineered Feature Summary

Feature	Rationale and Interpretation
<code>mileage_per_year</code>	Measures operational intensity rather than absolute mileage. Captures aggressive usage relative to vehicle age.
<code>engine_hours_per_km</code>	Indicates workload per distance traveled. High values reflect inefficient combustion or idle-heavy operations.
<code>fault_density</code>	Normalizes fault frequency relative to engine hours, capturing returning mechanical instability.
<code>thermal_stress</code>	Computed as oil temperature $\times$ engine load. Highlights compounded strain under operational loads.
<code>load_efficiency</code>	Ratio of engine load to fuel efficiency. Identifies elevated mechanical strain mapping dynamically to performance outputs.

4.3 Validation Through Feature Importance

After training tree-based models, parameter importance analysis confirmed that `thermal_stress`, `fault_code_count`, and `mileage_per_year` ranked among the most predictive variables, validating the engineering logic empirically.

5 Model Training and Selection Strategy

To evaluate model complexity reliably, a progressive approach established benchmarks across linear, rules-based, and ensemble frameworks.

5.1 Train-Test Split

The dataset was divided using an 80/20 train-test split via stratified sampling, preserving the minority maintenance class distributions perfectly within both cross-validation stages.

5.2 Progressive Model Evaluation

Logistic Regression (Baseline): Served as an interpretable linear benchmark. Though it achieved high baseline accuracy, its recall for the maintenance class remained low, proving the data’s underlying bounds were highly non-linear.

Decision Tree: Brought non-linear segmentation to the pipeline. While providing clear logical conditions, it notably suffered from high variance and reduced ROC-AUC, confirming overfitting.

Random Forest: Radically stabilized variances by bagging trees trained on sub-sampled data branches. It restored ROC-AUC levels impressively but still struggled with aggressive minority class identification.

XGBoost (Final Selected Model): Leveraging a rigorous gradient boosting strategy, this classifier corrected residual classification errors iteratively. Using integrated `scale_pos_weight` parameters, the framework naturally amplified loss gradients against false negatives, successfully capturing imbalanced targets with high stability.

5.3 Hyperparameter Optimization Strategy

Hyperparameters were strictly formulated via random searching (`RandomizedSearchCV`), focusing on balancing the precision-recall trade-off through 3-fold cross validation. Optimized constraints included:

- `n_estimators` and `max_depth`
- `subsample` and `learning_rate`
- `scale_pos_weight` (critical for combating the 75:25 dataset imbalance).

5.4 Model Comparison Summary

Table 3: Model Performance Comparison on Test Set

Model	Accuracy	Recall	F1-Score	ROC-AUC
Logistic Regression	0.84	0.60	0.68	0.92
Decision Tree	0.83	0.60	0.66	0.85
Random Forest	0.85	0.58	0.70	0.91
XGBoost (Tuned)	0.85	0.76	0.78	0.92

By successfully capturing a recall rate of 0.76 and ROC-AUC of 0.92, XGBoost was selected for system deployment.

6 Agentic Fleet Management Architecture

Transitioning from isolated predictive metrics to intelligent, prescriptive automation required embedding the serialized ML model within a sophisticated cognitive workflow. Located fundamentally within the `utils/` package architecture, the Agentic Fleet Management system orchestrates raw predictions alongside empirical diagnostic heuristics, context-aware semantic retrieval, and local generative processing.

6.1 Agent Components and State Graph Integration

The decision environment executes as a stateful, directed acyclic `StateGraph` managed securely via the LangGraph framework (`utils/langgraph_agent.py`). The agent injects predictive queries through a sequence of discrete operational nodes:

1. **Validation Node (`validate`):** Serves as the primary defensive barrier for the application. It parses explicit text variables—such as "abnormal vibrations" or "hot engine"—from operator maintenance prompts and forces corresponding values programmatically over the base telemetry dataset. From here, the integrated data passes to `validate_vehicle_input()` to guarantee all standardizations apply faithfully before scoring.
2. **Scoring Node (`score`):** Re-instantiates and executes the core XGBoost estimator algorithm (`utils/model_tool.py`). Upon calculating the raw algorithmic risk vectors, this node executes a rigid extraction loop testing the values across hard-coded subsystem thresholds. Output metrics directly derive tangible semantic signals, effectively mapping "risk_110_temp" directly into actionable definitions of "Thermal Stress" and subsequent component vulnerabilities.
3. **Retrieval Node (`retrieve`):** The engine uses Retrieval-Augmented Generation (RAG). Supported context paths within `utils/retriever.py` launch a localized semantic query searching the designated vector space. Relevant domain protocols matching the detected fault signals merge to define exact manuals and best-practice timelines needed by the vehicle operator.
4. **Reporting Node (`report`):** Operates as the synthesis endpoint (`utils/agent_logic.py`). Combining the quantitative risks and generated queries, the agent creates a programmatic `action_plan`. It formalizes timeline advice, assesses fleet policy compliance (SLA service requirements, lifecycle triggers), and structures the service outlook securely.

6.2 LLM Orchestration and Generative Enrichment

Crucially, the finalized report matrices process through a dynamic enrichment loop (`maybe_enrich_with`). Bridging external intelligence via local `ChatOllama` frameworks, the system refines structured technical payloads into consumable linguistic directives and conversational diagnostic responses.

By relying on LangGraph and empirical guardrails throughout the inference flow, risk probabilities generated by XGBoost bind tightly against factual system schemas, preventing standard generative hallucination gaps common in unmodified LLMs.

7 End-to-End System Workflow

The pipeline executes holistically, establishing an immediate feedback loop bridging physical vehicle telemetry with cognitive, automated dispatch intelligence. System integrity relies heavily on a tightly coupled software architecture.

7.1 Actual Execution Flow

1. **Streamlit Input:** Telemetry and free-form queries aggregate within localized front-end interfaces (`Home.py` / `pages/` / `app.py`).
2. **Feature Engineering and Preprocessing:** The `utils/preprocessor.py` matrix aligns schemas properly to internal vectors, enforcing encoding limits.
3. **Machine Learning Layer:** State nodes query the serialized `models/vehicle_model.joblib`.
4. **Graph Orchestration (LangGraph):** XGBoost probabilities route into `utils/langgraph_agent` triggering logic and rule-based diagnostic signals.
5. **Vector Retrieval (RAG):** `utils/retriever.py` filters relevant structural knowledge bases dynamically referencing the identified vulnerabilities.
6. **Generative Agent Delivery:** The reporting nodes bundle, format, LLM-enrich, and render comprehensive prescriptive fleet instructions seamlessly back to the UI.

8 Why This Architecture Matters

The main contribution of this project is that it does not stop at binary risk prediction. The deployed system combines a predictive classifier with a structured agent layer that explains the result, retrieves relevant maintenance context, and converts the prediction into a practical maintenance plan. This makes the system more useful for fleet operators than a raw probability score alone, because the output is interpretable, traceable, and action-oriented.

9 Agent Output Example

For a sample maintenance query such as *“This truck is overheating, vibrating, and overdue for service. What should be checked first?”*, the agent produces a structured response rather than a single label. The response includes a short risk summary, a prioritized action plan, retrieved maintenance context, and policy checks.

Table 4: Example of Agent Output Structure

Output Component	Example Content
Risk Summary	Maintenance required with a high-risk label based on the combined telemetry profile.
Action Plan	Inspect the cooling system, battery health, vibration sources, and overdue service items in priority order.
Retrieved Context	Relevant maintenance guidance from the knowledge base on overheating, repeated faults, and preventive service intervals.
Policy Checks	Flags for service overdue conditions, lifecycle review triggers, and route restriction under elevated risk.

10 Case Study: Direct Prediction vs Agent Interpretation

Using the same vehicle input, the direct predictor returns a numerical risk probability, while the agent turns that probability into a maintenance explanation. This distinction is important because a probability alone does not tell the user what subsystem to inspect first or why the vehicle is at risk. The agent output is therefore easier to interpret in operational settings, especially when multiple indicators contribute to the prediction.

In this project, the agent uses the same model output as the predictor, but it adds rule-based signals, retrieved documentation, and a structured response format. As a result, the final output is more useful for decision-making than a standalone score.

11 Implementation Summary

Table 5: Implemented Modules Summary

Module	Purpose
<code>utils/preprocessor.py</code>	Normalizes raw inputs, computes engineered features, and handles unknown fault-code values consistently.
<code>utils/model_tool.py</code>	Loads the trained model and produces prediction probability, class label, and risk score.
<code>pages/prediction.py</code>	Provides the interactive prediction interface for direct maintenance risk scoring.
<code>utils/langgraph_agent.py</code>	Orchestrates the validate, score, retrieve, and report nodes in the agent workflow.
<code>utils/retriever.py</code>	Retrieves maintenance context using keyword search or optional FAISS-based retrieval.
<code>utils/agent_logic.py</code>	Builds signals, action plans, policy checks, and the final structured agent response.
<code>test_agents.py</code> and <code>test_langgraph.py</code>	Validate the agent pipeline, output structure, and integration behavior.

12 Limitations and Future Work

- The model depends on fault-code count as an influential feature, so missing diagnostic information can still affect reliability.
- The dataset is structured and project-oriented, so future validation on real fleet data is necessary before production use.
- The current retriever is keyword-based by default, so broader semantic retrieval could improve context quality.
- Future work can include calibration, stronger uncertainty handling, and comparison against additional maintenance models.

13 Repository Architecture

To ensure sustainable production scaling, the repository segregates functional components deliberately:

Table 6: Project Directory and Component Layout

Path / Location	Primary Purpose and Alignment
data/	Contains processed training telemetry structures needed for standardizations.
models/	Secures the exact, serialized predictive machine learning artifacts (<code>.joblib</code>).
utils/	The core analytical backbone—houses <code>agent_logic.py</code> , <code>langgraph_agent.py</code> , <code>retriever.py</code> , <code>config.py</code> , bridging the gap between web UI and underlying cognitive modeling strings.
app.py / Home.py	Streamlit frontend execution protocols handling system-client network layers.
report/	Formal documentation frameworks ensuring technical transparency.
walkthrough/	Base research files representing experimental exploratory tests initially forming model baseline arguments.
assets/	Graphic styling utilities bridging interactive CSS variables required by the system interface.

14 Technology Stack

Table 7: Technology Stack Overview

Layer	Technology
Data Preprocessing & Pipeline	pandas, NumPy, scikit-learn
Model Generation Frameworks	XGBoost, RandomizedSearchCV
Agentic System Workflow	LangGraph, LangChain
Generative Intelligence & Retrieval	ChatOllama, keyword retrieval, optional FAISS
Deployment & Web Interface	Streamlit, HTML/CSS Integrations
System Persistence	joblib

15 Conclusion

Key Outcome: The project combines a tuned XGBoost risk model with a LangGraph-based agent to provide both maintenance prediction and structured decision support for fleet operations.

This implementation demonstrates how predictive maintenance can be extended into a practical decision-support system by linking telemetry-based prediction, contextual retrieval, and structured maintenance recommendations.

16 Team Contribution

The project was developed collaboratively with responsibilities distributed across model development, agent implementation, retrieval integration, and report preparation.

Table 8: Team Contribution Matrix

Team Member	Role and Responsibilities
Milind Bansal	Built the baseline and ensemble model pipelines. Deployed the agentic workflow in Streamlit. Validated the final prediction interface and outputs.
Isha Tomar	Conducted exploratory data analysis and feature engineering. Built the full LangGraph-based agent implementation. Integrated the end-to-end agent pipeline.
Yashi Agrawal	Developed the Streamlit deployment application. Implemented <code>agent_logic.py</code> and response generation flow. Refined the UI integration.
Mansi Agarwal	Wrote and formatted the final report. Implemented the RAG layer and retrieval prompts. Validated agent outputs and documentation structure.

17 References

- [1] Vehicle Maintenance Predictor & Agentic Platform – GitHub. https://github.com/11yashiagrwal/vehicle_maintenance_dashboard
- [2] LangGraph framework references for Stateful Execution mapping. <https://python.langchain.com/docs/langgraph>
- [3] Vehicle Maintenance Predictor – Live Deployment Target. <https://vehicle-maintenance-dashboard.streamlit.app/>